SLAM-Based Autonomous Drone for GPS-Denied Indoor Navigation

Project Work Log

Team: Kishan Ballal, Yathivandith JP | Platform: ROS 2 + Gazebo + PX4 (simulation) | Target completion: 21 Nov 2026

Day 1 — 21 October 2026: Environment Setup

Outcome: A GPU-accelerated simulation environment is running on the HP Omen laptop. Gazebo runs at a Real Time Factor (RTF) of about 100%, and the PX4 x500 drone successfully took off and landed in simulation.

1. Why we restarted the setup

An earlier attempt ran ROS 2, Gazebo and RTAB-Map on WSL2 and the drone simulation was very slow. The cause was graphics rendering falling back to the CPU (software renderer "llvmpipe") instead of the GPU. When the simulator cannot keep up, the whole world runs in slow motion (Real Time Factor below 1).

We decided to rebuild from scratch on a capable laptop and to verify GPU rendering before installing anything heavy.

2. Hardware

Component	Specification
Laptop	HP Omen (HyperX)
CPU	Intel Core i7
RAM	24 GB
GPU	NVIDIA GeForce RTX 5060 Laptop GPU, 8 GB VRAM
Storage	~1 TB SSD (C: drive)

3. Platform decisions

We compared several options before choosing the stack:

- **Native Ubuntu dual-boot:** fastest, but requires repartitioning the drive. We chose not to modify the Windows installation.
- **WSL2 with GPU acceleration (chosen):** Ubuntu runs inside Windows, and with the right settings it renders on the RTX 5060. No partitioning needed.
- **NVIDIA Isaac Sim:** photorealistic, but its minimum requirement is an RTX 4080 with 16 GB VRAM. Too risky for an 8 GB GPU and a 4-week timeline.
- **MATLAB/Simulink UAV Toolbox:** good for controller design, weak for ROS 2 autonomy and frontier exploration.
- **AirSim / Webots:** AirSim is archived; Webots lacks proper PX4 integration.

4. Software stack

Layer	Choice	Purpose
Host OS	Windows 11 + WSL 3.0.3	Runs Linux inside Windows

Layer	Choice	Purpose
Linux OS	Ubuntu 24.04 LTS	PX4's primary supported platform
GPU driver	NVIDIA 621.01 (Windows)	WSL uses the Windows driver directly
Middleware	ROS 2 Jazzy (LTS)	Robot software framework
Simulator	Gazebo Harmonic	Physics + sensor simulation (official pairing with Jazzy)
Flight controller	PX4 SITL (stable release)	Simulated drone autopilot
Drone model	x500 quadcopter	Standard PX4 Gazebo drone

Planned for later milestones: Micro-XRCE-DDS Agent (PX4 to ROS 2 bridge), slam_toolbox, RTAB-Map / nvblox, Nav2, explore_lite, rf2o laser odometry, YOLO, Foxglove Studio.

5. Steps completed

Step 1 — NVIDIA driver (Windows)

Updated the driver through the NVIDIA App (Drivers → Download → Express Installation), then restarted. Verified in PowerShell:

```
nvidia-smi
```

Result: NVIDIA GeForce RTX 5060, driver 621.01, 8151 MiB memory.

Step 2 — Install WSL2 and Ubuntu 24.04

In an administrator PowerShell:

```
wsl --update
wsl -l -v
wsl --install -d Ubuntu-24.04
```

WSL updated to version 3.0.3 and Ubuntu 24.04 LTS was installed. A Linux user account was created on first launch.

Step 3 — Increase memory available to WSL

By default WSL only receives half of system RAM. Created C:\Users\kisha\.wslconfig:

```
notepad $env:USERPROFILE\.wslconfig

[wsl2]
memory=16GB
swap=8GB

wsl --shutdown
```

Verified inside Ubuntu with free -h: 15 GiB memory and 8 GiB swap available.

Step 4 — Verify GPU inside Ubuntu

```
sudo apt update && sudo apt upgrade -y
nvidia-smi
sudo apt install -y mesa-utils
glxinfo -B | grep "OpenGL renderer"
```

nvidia-smi detected the RTX 5060, but the OpenGL renderer showed "llvmpipe" (CPU rendering). The laptop has both Intel and NVIDIA GPUs, so we forced the Direct3D 12 driver to use the NVIDIA GPU:

```
echo 'export GALLIUM_DRIVER=d3d12' >> ~/.bashrc
echo 'export MESA_D3D12_DEFAULT_ADAPTER_NAME=NVIDIA' >> ~/.bashrc
source ~/.bashrc
glxinfo -B | grep "OpenGL renderer"
```

Result: "D3D12 (NVIDIA GeForce RTX 5060 Laptop GPU)" — graphics now render on the GPU. This was the key fix for the earlier slowness.

Step 5 — Install ROS 2 Jazzy and Gazebo Harmonic

```
sudo apt install -y software-properties-common curl
sudo add-apt-repository -y universe
# add the ROS 2 apt source (ros2-apt-source package), then:
sudo apt update && sudo apt upgrade -y
sudo apt install -y ros-jazzy-desktop ros-dev-tools ros-jazzy-ros-gz
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

Step 6 — Simulation speed test

```
gz sim shapes.sdf
```

Result: RTF 99.92% — the simulator runs at full real-time speed, with shadows and full rendering.

Step 7 — Install and build PX4

```
cd ~
git clone https://github.com/PX4/PX4-Autopilot.git --recursive
cd PX4-Autopilot
git checkout <latest stable release tag>
git submodule update --init --recursive
bash ./Tools/setup/ubuntu.sh --no-nuttx
make px4_sitl gz_x500
```

The --no-nuttx flag installs only simulation dependencies (no flight-hardware toolchain). Gazebo opened with the x500 drone at 100% RTF.

Step 8 — First flight

Takeoff was initially refused with "Preflight Fail: No connection to the GCS", because PX4 by default requires a Ground Control Station (e.g. QGroundControl). For simulation we disabled the data-link-loss failsafe and the RC requirement at the pxh> prompt:

```
param set NAV_DLL_ACT 0
param set COM_RC_IN_MODE 4
commander takeoff
commander land
shutdown
```

Result: the drone took off to about 2.5 m, hovered, and landed successfully. The parameters are saved and persist between sessions.

6. Verification summary

Check	Result	Status
NVIDIA driver (Windows)	621.01, RTX 5060 detected	Pass
GPU visible in Ubuntu	nvidia-smi shows RTX 5060	Pass
OpenGL renderer	D3D12 (NVIDIA RTX 5060)	Pass (after fix)
WSL memory	15 GiB RAM + 8 GiB swap	Pass
Gazebo speed test	RTF 99.92%	Pass
PX4 x500 simulation	Loads at 100% RTF	Pass
Takeoff and landing	Successful	Pass (after parameter fix)

7. Issues encountered and fixes

Issue	Cause	Fix
Earlier sim was very slow	CPU rendering (llvmpipe) under WSL2	Updated NVIDIA driver; forced d3d12 + NVIDIA adapter in ~/.bashrc
WSL had only ~12 GB RAM	WSL default is 50% of system RAM	.wslconfig with memory=16GB, swap=8GB
Gazebo bottom bar hidden	Window taller than screen	Maximised the Gazebo window
"Preflight Fail: No connection to the GCS"	PX4 requires a ground station by default	param set NAV_DLL_ACT 0; param set COM_RC_IN_MODE 4
gz_frame_id XML warnings	Harmless SDF warnings from the PX4 model	None needed
BitLocker turned off during dual-boot prep	Encryption disabled while preparing to repartition	Re-enabled Device Encryption; verified 100% encrypted, protection on

8. How to relaunch the simulation

1. Open "Ubuntu 24.04" from the Windows Start menu.
2. Run the commands below. Gazebo opens with the drone in under a minute.

```
cd ~/PX4-Autopilot
make px4_sitl gz_x500
```

3. Wait about 15 seconds, then type commander takeoff at the pxh> prompt.
4. To stop: commander land, then shutdown (or Ctrl + C).

Everything is stored permanently in the WSL virtual disk on the C: drive and survives laptop restarts.

Milestone Plan and Status

The project is built in layers. Each milestone must be demoable on its own, is frozen with a git tag and demo video, and has its own launch file so earlier demos never break.

Milestone	Dates	Deliverable	Status
M0	21–25 Oct	Drone in an indoor world with live sensor dashboard (LiDAR + depth camera), teleop flight	In progress — setup and first flight done
M1	2–8 Nov	Waypoint flight with live 2D (slam_toolbox) and 3D (RTAB-Map) mapping	Not started
M2	9–15 Nov	GPS-denied flight (rf2o odometry into PX4) + autonomous frontier exploration	Not started
M3	16–21 Nov	YOLO victim detection on the map, nvblox 3D mesh, final video and report	Not started

Note: No project work from 26 Oct to 1 Nov (internal exams).

Next steps (remaining Milestone 0)

1. Connect PX4 to ROS 2: Micro-XRCE-DDS Agent + px4_msgs workspace.
2. Drone model with a 2D LiDAR and a depth camera.
3. Indoor world: 3–4 rooms, a corridor and a loop.
4. Keyboard teleop and a Foxglove dashboard showing live sensor data.